

CONTENTS

MAIN

1. What Bolyra is
2. Pick your path
3. Day 1 — Setup
4. Week 1 — First PR
5. Month 1 — Own an area

APPENDICES

- A. Circuits
 - B. Contracts
 - C. SDK (TS + Python)
 - D. Integrations
 - E. Culture & workflow
 - F. Gotchas cheat-sheet
 - G. Quick reference
-

Welcome to Bolyra

Engineer onboarding guide — from `git clone` to first merged PR.

This is the entry point for anyone new to the Bolyra codebase — a future teammate, an external open-source contributor, or a specialist brought in to work on circuits or contracts. It takes you from `git clone` to your first merged PR, and points you at the canonical reference docs (`CLAUDE.md` , `tasks/lessons.md` , `circuits/FORMAL-PROPERTIES.md` , etc.) for everything else.

Only have ten minutes? Read §1, §2, and Appendix F.

1. What Bolyra is (60-second pitch)

Bolyra is a **mutual zero-knowledge proof identity protocol** for humans and AI agents.

- **Humans** prove uniqueness via a Semaphore v4-style enrollment circuit (Groth16).
- **Agents** prove EdDSA-signed credentials with cumulative-bit permissions (Groth16 + PLONK).
- A one-way **delegation** circuit narrows scope; it can never expand it.
- A **handshake** binds both proofs to a shared `sessionNonce` and is verified atomically on-chain.

Current scope is **Phase 1 — Proof of Enrollment**. Delegation is a v0.3 stub at the SDK layer; circuits and contracts for it are live and tested. Framework integrations (LangChain, CrewAI, OpenClaw, MCP, x402) have a stable API shape but are stubs pending SDK v0.2 wiring.

- **Domain:** bolyra.ai
- **Company:** ZKProva Inc.
- **License:** Apache-2.0 with DCO sign-off on every commit.
- **Patent:** US provisional #64/043,898 (filed 2026-04-20). Apache 2.0 Section 3 patent grant applies to every contribution.

For the marketing-free version, see `README.md` and the architecture section of `CLAUDE.md`.

2. Pick your path

If you are...	Read
Joining the team / first hire	Everything below, in order.
Sending a single OSS PR	§3 (Day 1), §4 PR workflow, Appendix F (gotchas).
ZKP specialist scoped to circuits	§3, then Appendix A. Skim Appendix F.
Smart-contract engineer	§3, then Appendix B. Skim Appendix F.
SDK engineer (TS or Python)	§3, then Appendix C.
Integrations engineer	§3, then Appendix D.

Appendices E (culture/workflow) and F (gotchas cheat-sheet) apply to everyone.

3. Day 1 — Environment & first build (~2 hours)

Goal: end the day with `npm test` green on your laptop and DCO sign-off configured.

3.1 Prerequisites

Tool	Version	Notes
Node.js	18+ (20+ recommended)	Used by SDK, circuits, contracts.
Python	3.11+	Used by <code>sdk-python/</code> .
circom	2.x	Install separately (not on npm); see docs.circom.io .
git	any modern	Must support <code>git commit -s</code> .

macOS: `brew install node python circom`. Linux: package manager + a prebuilt circom binary from the circom release page.

3.2 Clone and install (per-workspace)

The repo is not an npm workspace. You must install dependencies in each subpackage independently:

```
git clone https://github.com/bolyra/bolyra.git
cd bolyra

npm install                                # root: orchestration only
( cd sdk      && npm install )
( cd circuits && npm install )
( cd contracts && npm install )
( cd sdk-python && pip install -e ".[dev]" )
```

Most common Day 1 failure. If you skip a subpackage install, the root `npm test` will fail with a missing-binary error from inside `cd circuits` or `cd contracts`.

3.3 Compile circuits and contracts

```
npm run compile:circuits      # ~1 min the first time
npm run compile:contracts    # ~30 s
```

`compile:circuits` writes artifacts (`.r1cs` , `.zkey` , `.wasm` , `.vkey.json` , `pot16.ptau`) to `circuits/build/` . The Powers of Tau file is ~75 MB. `compile:contracts` regenerates Solidity verifiers from the current `.zkey` files — keep this in mind any time a circuit changes (see Appendix B).

3.4 Run the test suite

```
npm test                      # fast circuits + contracts (~2 min)
npm run test:circuits:fast    # witness-gen only (~30 s)
FULL_PROOF=1 npm run test:circuits:slow # full Groth16/PLONK (~2 min)
npm run test:contracts        # Hardhat
( cd sdk      && npm test )    # Jest
( cd sdk-python && pytest -v )
```

CI runs the fast path and the SDK suites. Circuit and contract tests are **not** run in CI (the Powers of Tau file is too large). They must pass locally before you push.

Acceptance: `npm test` exits 0.

3.5 Configure DCO sign-off once

Every commit needs `Signed-off-by:` . CI rejects unsigned commits and there is no way around it.

```
git config user.name "Your Name"
git config user.email "you@example.com"

# Sign off every commit
git commit -s -m "fix: handle empty proof input"

# Fix the most recent commit if you forgot
git commit --amend -s --no-edit

# Backfill a whole branch
git rebase -i --signoff <base-branch>
```

GPG signing (`-S`) is encouraged and **required** for commits forwarded upstream to x402. See [CONTRIBUTING.md](#) for the full DCO text and GPG setup.

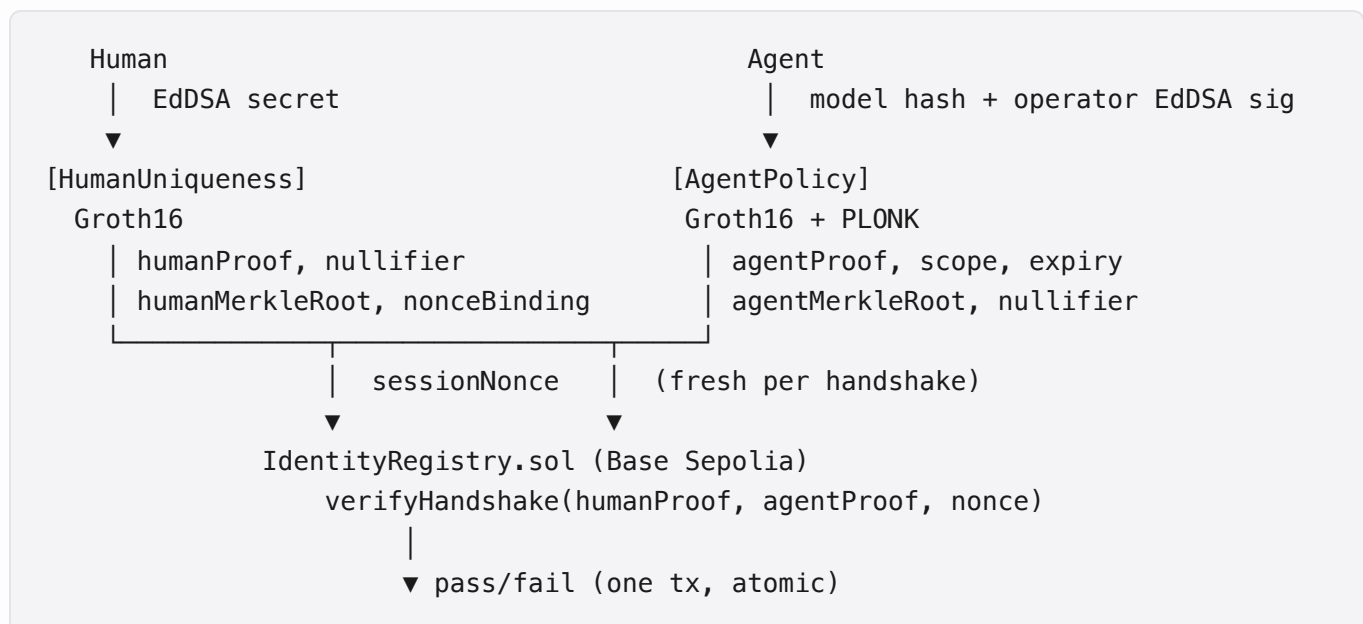
3.6 If something fails

Symptom	Likely cause
<code>npx: command not found</code> inside <code>circuits/</code> or <code>contracts/</code>	Skipped a per-workspace <code>npm install</code> .
Circuit tests fail with "could not find <code>.zkey</code> "	<code>npm run compile:circuits</code> not run.
Solidity verifier reverts on a known-good proof	Verifier out of sync with current <code>.zkey</code> . Re-run compile then <code>npm run compile:contracts</code> .
CI fails with "DCO check failed"	Missing <code>Signed-off-by:</code> on one or more commits. Amend / rebase.
Hardhat complains about <code>BASE_SEPOLIA_RPC_URL</code>	Only needed for <code>deploy:base-sepolia</code> ; not for <code>test</code> .

Full lessons-learned in [tasks/lessons.md](#) — skim it once today and reread it any time something surprises you.

4. Week 1 — Mental model & first PR (3–5 days)

4.1 The protocol in one diagram



`Delegation.circom` lives off to the side: a delegator runs it to issue a narrowed credential to a delegatee. The circuit enforces "narrow only, never expand."

4.2 Module map

Path	Role
<code>circuits/</code>	Circom 2 circuits + snarkjs (dev) / rapidsnark (prod) proving.
<code>contracts/</code>	Hardhat — Solidity verifier contracts + <code>IdentityRegistry</code> .
<code>sdk/</code>	<code>@bolyra/sdk</code> — TypeScript public API.
<code>sdk-python/</code>	<code>bolyra</code> Python SDK — pure types + subprocess bridge to Node.
<code>integrations/</code>	LangChain, CrewAI, OpenClaw, MCP, payment-protocols (x402).
<code>spec/</code>	DID method, IETF-style draft, conformance runner, test vectors.
<code>examples/</code>	<code>mcp-demo</code> , <code>provider-mock</code> , <code>quickstart.ts</code> .
<code>docs/</code>	Public docs (quickstart, OWASP agentic mapping).
<code>*-autoresearch/</code>	Four independent research loops (discovery, differentiation, patent, protocol). Don't cross-pollinate.

4.3 Reading order (time-budgeted)

1. `README.md` — 10 min.
2. `CLAUDE.md` — 20 min. Architecture, commands, gotchas.
3. `sdk/QUICKSTART.md` — 15 min. Hands-on.
4. `docs/quickstart.md` — 10 min. Repo-level quickstart; overlaps with the SDK one.
5. `spec/draft-bolyra-mutual-zkp-auth-01.md` §1–§4 — 30 min. Wire format and nonce binding.
6. `tasks/lessons.md` — skim, 20 min. Reread when blocked.

Total: ~2 hours of reading.

4.4 Your first PR

Browse `tasks/todo.md` for current sprint items. Good starter tickets typically look like:

- Doc fixes or clarifications.

- Adding test vectors to `spec/test-vectors.json`.
- Implementing one of the integration stubs in `integrations/`.
- Filling in a missing type or test in `sdk/`.

PR workflow:

1. Branch from `main`. Atomic commits — one logical change per commit.
2. Run the local test suite (`npm test` , plus area-specific tests).
3. If you touched TypeScript: (`cd sdk && npm run typecheck`). CI gates on it.
4. `git commit -s -m "..."` for every commit.
5. Push, open PR with a clear motivation paragraph.
6. The DCO action verifies sign-offs; CI must go green.
7. The `/pre-ship` workspace command (when used) spawns reviewers based on the diff: `circuit-auditor` , `sdk-api-reviewer` , `protocol-reviewer` . Trust the gate — don't skip it.

See `CONTRIBUTING.md` for the canonical workflow.

Acceptance: one PR open, DCO and CI green, reviewer assigned.

5. Month 1 — Pick an area to own

By Month 1 you should be able to own work end-to-end in one of the areas below. Pick the one closest to your background and use the matching appendix as a deep-dive.

Area	Looks like	Appendix
Circuits	New circuits, optimizing constraints, ceremony work, fuzzing inputs.	A
Smart contracts	Registry features, verifier regen, on-chain gas tuning, multichain.	B
SDK (TS / Python)	Public API shape, ergonomics, framework adapters, error model.	C
Integrations	LangChain/CrewAI/MCP/OpenClaw/x402 adapters, conformance tests.	D

Appendix A — Circuits deep-dive

Stack: Circom 2, snarkjs (dev/test), rapidsnark (prod), Mocha tests.

Circuits:

Circuit	Proving system	Notes
<code>HumanUniqueness</code>	Groth16 only	Reuses public Semaphore v4 ceremony at depth 20. Don't regenerate it.
<code>AgentPolicy</code>	Groth16 + PLONK	Dual-build. Both <code>.zkey</code> ship.
<code>Delegation</code>	Groth16 + PLONK	Dual-build. Enforces one-way scope narrowing.

Trusted setup: `pot16.ptau` (Hermes Phase 1, $2^{16} \approx 65k$ constraints) is the universal SRS for project-specific Groth16 keys. If a circuit grows past that, bump to `pot17` and flag in PR review.

Test split:

- `npm run test:circuits:fast` — witness-generation only, ~30 s. Default in CI.
- `FULL_PROOF=1 npm run test:circuits:slow` — full Groth16/PLONK, ~2 min. Gate on a label, not the default.

Required reading:

- [circuits/FORMAL-PROPERTIES.md](#) — soundness, completeness, privacy properties in formal notation.
- [circuits/CEREMONY.md](#) — trusted setup pipeline.

Top circuit gotchas:

- Solidity verifier must match current `.zkey`. After any circuit change, re-run `npm run compile:circuits` and `npm run compile:contracts`. Unit tests can pass against a stale verifier; the Hardhat `verifyProof` integration tests catch it.

- `rapidsnark` is production, `snarkjs` is dev/test. Don't ship snarkjs paths into production code.
- After changing a dual-build circuit, verify both Groth16 and PLONK verifier contracts still match.

Appendix B — Contracts deep-dive

Stack: Hardhat + ethers.js v6. Target chain: Base Sepolia (chain ID 84532).

Deployed addresses: see `contracts/deployments/base-sepolia.json`. The registry is `IdentityRegistry`; verifier contracts are `Groth16Verifier`, `PlonkVerifier`, `DelegationPlonkVerifier`. Linked Poseidon library is `PoseidonT3`. Treat the JSON file as the source of truth — don't copy addresses into other docs.

Deploy commands:

```
( cd contracts && npm run deploy:local )      # local hardhat node
( cd contracts && npm run deploy:base-sepolia )  # testnet
```

Testnet deploy requires `BASE_SEPOLIA_RPC_URL` and `DEPLOYER_PRIVATE_KEY` in a `contracts/.env` file. Never commit `.env`.

Verifier regen checklist (after any circuit change):

1. `npm run compile:circuits` — regenerates `.zkey`, `.vkey.json`.
2. Regenerate the Solidity verifier(s) from the new `vkey.json`.
3. `npm run compile:contracts` — recompile.
4. `npm run test:contracts` — confirm the verifier accepts a freshly generated proof.
5. If deploying, redeploy the verifier and update `IdentityRegistry` to point at the new address.

Appendix C — SDK deep-dive (TS + Python)

TypeScript SDK (`sdk/`, `@bolyra/sdk`):

- Public API: `createHumanIdentity(secret)`, `createAgentCredential(modelHash, operatorPrivKey, permissions, expiry)`, `proveHandshake(human, agent)`, `verifyHandshake(humanProof, agentProof, nonce)`.
- Delegation API is v0.3 stub.
- Treat any change to these signatures as breaking. Run the `sdk-api-reviewer` subagent (via `/pre-ship`) on SDK changes.
- Strict TS, no `any` without an inline justification comment.
- `( cd sdk && npm run typecheck )` is a CI gate. Run it before every push.

Python SDK (`sdk-python/`, `bolyra`):

- Pure-Python types and validation only. All proving spawns the Node `@bolyra/sdk` as a subprocess (snarkjs is JS-only).
- When adding a Python feature that needs proving, expose it through the subprocess bridge. **Do not** reimplement crypto in Python.
- Tests: `( cd sdk-python && pytest -v )`.

Permission model — 8-bit cumulative encoding, higher tiers imply lower. The full table is in the "Permissions Model" section of [CLAUDE.md](#). `validateCumulativeBitEncoding()` enforces the implication rules in the SDK; the `Delegation` circuit enforces them on-chain. Don't bypass either.

Appendix D — Integrations deep-dive

Adapters live under [integrations/](#):

- `langchain/` — LangChain tools (`BolyraAuthTool`, `BolyraDelegateTool`).
- `crewai/` — CrewAI agent wrappers.
- `openclaw/` — OpenClaw agent framework adapter. Also type-checked in CI (peer dep on `@bolyra/sdk`).
- `mcp/` — Model Context Protocol server. Built artifact: `examples/mcp-demo/dist/bolyra-proxy.js`, registered via `.mcp.json`. Requires `BOLYRA_RAPIDSNAK` env pointing at `circuits/build/rapidsnark_prover`.
- `payment-protocols/` — x402 v2 conformance. Has its own wire-format conformance suite.

Status: API shape is final; most implementations are stubs pending SDK v0.2 wiring. Good area for first contributions.

Conformance runner: `node spec/conformance-runner.js` against `spec/test-vectors.json`.

Appendix E — Culture & workflow

These are workspace norms, not optional preferences:

- **Plan first for non-trivial work** (3+ steps or any architectural decision). Write the plan, get agreement, then execute.
 - **Test-first bug fixing.** Write a failing reproducing test before the fix. The test stays in the suite.
 - **Verify before declaring done.** Run tests, check output, prove correctness. Don't mark a task complete without evidence.
 - **Update `tasks/lessons.md` after corrections.** Write the rule that prevents the same mistake. Lessons file is the long-memory of the project.
 - **Subagents for research and parallel work.** Keeps the main context clean; one task per subagent for focused execution.
 - **/pre-ship is mandatory for non-trivial PRs.** It spawns `circuit-auditor`, `sdk-api-reviewer`, or `protocol-reviewer` based on what changed.
 - **Simplicity bias.** Make the smallest change that solves the problem. Avoid premature abstraction; three similar lines beat a one-time helper.
-

Appendix F — Gotchas cheat-sheet (one page)

Distilled from `CLAUDE.md` and `tasks/lessons.md`. One-liners; follow the link for full context.

- **Every commit needs** `git commit -s`. CI rejects unsigned. Fix with `git commit --amend -s --no-edit`.
- **Repo is `bolyra/`, not `identityos/`.** Legacy name survives only in historical patent drafts (`drafts/IDENTITYOS-PROV-001-*`).

- **License is uniformly Apache-2.0.** Anything saying "MIT" is stale — fix on sight.
- **Per-workspace `npm install` is required.** Root only orchestrates.
- **Circuit + contract tests are SKIPPED in CI.** Run them locally before push.
- **Don't regenerate the Semaphore v4 ceremony.** `HumanUniqueness` reuses the public ceremony at depth 20.
- **`pot16.ptau` caps at ~65k constraints.** Bump to `pot17` if you outgrow it; flag in PR review.
- **Solidity verifier must match the current `.zkey`.** Regenerate the verifier after any circuit change.
- **`rapidsnark` is prod, `snarkjs` is dev/test.** Don't ship snarkjs paths to production.
- **Dual-build circuits ship both Groth16 and PLONK `.zkey`.** Verify both verifiers after any change to `AgentPolicy` or `Delegation`.
- **Scope narrowing in delegation is one-way.** Enforced in `Delegation.circom`, not just the SDK. Don't add SDK-level shortcuts.
- **Handshake nonce binding is non-negotiable.** Every handshake commits to a fresh `sessionNonce`. Replaying without rebinding fails by design.
- **Permission encoding is cumulative.** Higher tiers imply lower. Don't bypass `validateCumulativeBitEncoding()` or the `Delegation` circuit.
- **Python SDK is a thin shell.** All proving spawns Node. Don't reimplement crypto in Python.
- **TS SDK public API changes are breaking.** Run `sdk-api-reviewer`.
- **Run `( cd sdk && npm run typecheck )` before pushing TS changes.** CI gates on it.
- **Don't mix winners across the four autoresearch loops.** Scoring rubrics differ.
- **`.mcp.json` paths must be absolute and correct.** `bolyra-fs` fails silently on a wrong path; needs `BOLYRA_RAPIDSNARK` env.
- **`settings.local.json` is per-user / gitignored.** Don't paste cross-project permissions there.

Appendix G — Quick reference (one screen)

Build & test

```

npm install                                # root (orchestration)
( cd sdk      && npm install )
( cd circuits && npm install )
( cd contracts && npm install )
( cd sdk-python && pip install -e ".[dev]" )

npm run compile:circuits                   # circuits/build/
npm run compile:contracts                  # contracts artifacts

npm test                                  # fast circuits + contracts
npm run test:circuits:fast                 # witness-gen only
FULL_PROOF=1 npm run test:circuits:slow    # full proofs
npm run test:contracts                     # Hardhat
( cd sdk      && npm test )                 # Jest
( cd sdk      && npm run typecheck )        # CI gate
( cd sdk-python && pytest -v )

```

Deploy (contracts)

```

( cd contracts && npm run deploy:local )
( cd contracts && npm run deploy:base-sepolia )    # needs .env

```

Commit

```

git commit -s -m "... "                   # DCO sign-off
git commit -s -S -m "... "                # DCO + GPG (upstream-bound)
git commit --amend -s --no-edit            # fix missing sign-off

```

Key files

- [README.md](#) , [CLAUDE.md](#) , [CONTRIBUTING.md](#)
- [sdk/QUICKSTART.md](#) , [docs/quickstart.md](#)
- [circuits/FORMAL-PROPERTIES.md](#) , [circuits/CEREMONY.md](#)
- [spec/draft-bolyra-mutual-zkp-auth-01.md](#) , [spec/did-method-bolyra.md](#)
- [contracts/deployments/base-sepolia.json](#)
- [tasks/lessons.md](#) , [tasks/todo.md](#)
- [docs/owasp-agentic-mapping.md](#)

External

- Website: <https://bolyra.ai>
- npm: `@bolyra/sdk`
- PyPI: `bolyra`
- Chain: Base Sepolia (84532), RPC `https://sepolia.base.org`
- Repo: `github.com/bolyra/bolyra`

Found something missing or wrong? File a PR against this doc — it's the kind of starter contribution we welcome.